



Lab 1 - Setting Up the Course Lab Environment

Overview

Before we dive into the hands-on activities, let's configure the lab environment with the following tools:

- **Docker:** Used by kind to provision the Kubernetes cluster.
- **kind:** Used to create Kubernetes clusters.
- **kubecttl:** Used to manage the Kubernetes cluster.
- **Helm:** Used for Kubernetes package management.
- **Siege:** A load testing tool.

Exercise 1.1: Set Up the Course Lab Environment

Linux Environment Setup

While we present various setup options in this section, we will use an Ubuntu 22.04 host and a Kubernetes cluster setup using **kind**.

To set up **Docker**, follow these steps.

1. Install Docker:

```
curl -fsSL https://get.docker.com/ | sh
```

2. Enable Docker to start on boot:

```
sudo systemctl enable --now docker
```

3. Check that the Docker service is running:

```
sudo systemctl status docker
```

4. Add current user to Docker group:

```
sudo usermod -aG docker $USER
```

5. Refresh shell session (**by exiting & logging in again**).

6. Verify Docker installation:

```
docker ps
```

To install the latest version of **kubect1**, execute the commands below.

1. Download the binary:

```
curl -sSL -O "https://dl.k8s.io/release/$(curl -L -s  
https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubect1"
```

2. Modify permissions:

```
chmod +x kubect1
```

3. Move to `/usr/local/bin`:

```
sudo mv kubect1 /usr/local/bin
```

To install the latest **Helm** version, perform the following steps.

1. Download Helm (if the link doesn't work, update the link with a version from [GitHub releases](#)):

```
curl -sSL -O https://get.helm.sh/helm-v3.13.0-linux-amd64.tar.gz
```

2. Extract the downloaded archive:

```
tar -xzf helm-v3.13.0-linux-amd64.tar.gz
```

3. Move the Helm binary to `/usr/local/bin`:

```
sudo mv linux-amd64/helm /usr/local/bin/helm
```

To install the latest version of **Siege**, execute the following command.

1. Install the binary:

```
sudo apt-get install siege
```

Siege is a multi-threaded HTTP/FTP load tester and benchmarking utility.

To install **kind**, run the following command.

1. For AMD64 / x86_64:

```
[ $(uname -m) = x86_64 ] && curl -Lo ./kind  
https://kind.sigs.k8s.io/dl/v0.20.0/kind-linux-amd64
```

2. For ARM64:

```
[ $(uname -m) = aarch64 ] && curl -Lo ./kind  
https://kind.sigs.k8s.io/dl/v0.20.0/kind-linux-arm64
```

3. Modify permissions:

```
chmod +x ./kind
```

4. Move the kind binary `/usr/local/bin`:

```
sudo mv ./kind /usr/local/bin/kind
```

5. Create cluster:

```
kind create cluster
```

```
Creating cluster "kind" ...
```

```
✓ Ensuring node image (kindest/node:v1.27.3) 
```

```
✓ Preparing nodes 
```

```
✓ Writing configuration 
```

```
✓ Starting control-plane 
```

```
✓ Installing CNI 
```

```
✓ Installing StorageClass 
```

```
Set kubectl context to "kind-kind"
```

```
You can now use your cluster with:
```

```
kubectl cluster-info --context kind-kind
```

6. Verify cluster:

```
kubectl get ns
```

NAME	STATUS	AGE
default	Active	36s
kube-node-lease	Active	36s
kube-public	Active	36s
kube-system	Active	36s
local-path-storage	Active	30s

Metric Server Setup in Kubernetes

To install metrics server in Kubernetes, execute the commands below.

1. Add Helm repository:

```
helm repo add metrics-server https://kubernetes-sigs.github.io/metrics-server/
```

2. Install metrics server:

```
helm upgrade --install metrics-server metrics-server/metrics-server -n
kube-system --set args[0]=--kubelet-insecure-tls
```

Release "metrics-server" does not exist. Installing it now.

NAME: metrics-server

LAST DEPLOYED: Tue Mar 5 11:20:36 2024

NAMESPACE: kube-system

STATUS: deployed

REVISION: 1

TEST SUITE: None

NOTES:

```
*****
* Metrics Server *
*****
Chart version: 3.12.0
App version: 0.7.0
Image tag: registry.k8s.io/metrics-server/metrics-server:v0.7.0
```

3. Verify installation:

```
kubectl get pods -n kube-system -l=app.kubernetes.io/name=metrics-server
```

NAME	READY	STATUS	RESTARTS	AGE
metrics-server-55fd7d4789-v6dpw	1/1	Running	0	63s

Congratulations, your environment is ready!



Training & Certification

Lab 2 - Implementing HPA in Kubernetes

Overview

In this lab, you will gain hands-on experience with Kubernetes autoscaling by setting up Horizontal Pod Autoscaler (HPA) for a deployment. You will generate a load to see the effects of autoscaling in action.

Prerequisites

Kubernetes cluster with metric server installed as per Lab 1.

Exercise 2.1: Deploy Sample Application in Kubernetes

In this first exercise, we will deploy a sample application in Kubernetes. This application would be used to demonstrate the HPA autoscaling strategies that are available in Kubernetes.

1. Create **webapp.yaml** file with contents below. This file defines a Kubernetes deployment for your sample application:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: webapp
spec:
  replicas: 2
  selector:
    matchLabels:
      app: webapp
  template:
    metadata:
      labels:
        app: webapp
    spec:
      containers:
        - image: nginx
          name: nginx
          resources:
```

```
limits:
  cpu: "10m"
requests:
  cpu: "10m"
```

2. Deploy the application:

```
kubectl apply -f webapp.yaml
```

3. Verify the deployment:

```
kubectl get deployments
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
webapp	2/2	2	2	89s

4. Set up port forwarding to access the application.

In this step, we are establishing a port forwarding rule that redirects network traffic from a specific port on your local machine to the corresponding port on the Kubernetes pod hosting the server. By doing so, you enable direct access to the server via `http://localhost:8080` from your local computer. This action bridges the network gap between your local environment and the isolated Kubernetes pod, allowing you to test and interact with the deployed application as if it were running locally. **It's important to run this in a new terminal tab to keep the port forwarding active throughout your testing session.**

```
kubectl port-forward deploy/webapp 8080:80
```

```
Forwarding from 127.0.0.1:8080 -> 80
Forwarding from [::1]:8080 -> 80
```

5. Test the sample application by sending an HTTP request:

```
curl http://localhost:8080/
```

```
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
<Output Truncated>
```

Exercise 2.2: Configure & Test HPA

In this exercise, we will configure the Horizontal Pod Autoscaler in Kubernetes.

1. Create an HPA resource.

This resource will autoscale your deployment based on CPU utilization.

We are starting off by creating a Horizontal Pod Autoscaler for our Kubernetes deployment using the `kubectl autoscale` command:

```
kubectl autoscale deployment webapp --min=2 --max=5 --cpu-percent=20
```

```
horizontalpodautoscaler.autoscaling/webapp autoscaled
```

2. Check that HPA is correctly deployed:

```
kubectl get hpa
```

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	AGE
webapp	Deployment/webapp	0%/20%	2	5	2	23s

3. Generate load using Siege.

In this step, we are using the **Siege** tool to generate traffic for our application. As CPU usage surpasses the set thresholds, the HPA responds automatically by triggering its scaling mechanism. The HPA scales up the number of pod replicas to handle the increased load. This scaling up ensures that the application maintains its performance and responsiveness despite the heightened demand.

```
siege -q -c 2 -t 1m http://localhost:8080
```

4. Monitor autoscaling.

In this step, we are actively monitoring the behavior of the Horizontal Pod Autoscaler as it responds to the increased load. By executing `kubectl get hpa`, you can observe the HPA's real-time response to changing CPU utilization. This command displays crucial metrics such as current CPU usage, targeted usage (set at 20% in our configuration), and the current number of replicas.

HPA operates by continually monitoring specified metrics - in this case, CPU utilization - and adjusting the number of pod replicas to meet the desired target utilization. When the load increases and CPU usage rises above the 50% threshold, the HPA responds by scaling up the number of pods.

```
kubectl get hpa -w
```

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	AGE
webapp	Deployment/webapp	20%/20%	2	5	3	85s

The above output shows that HPA has scaled the **webapp** deployment to 3 replicas. This scaling action is a direct response to increased CPU utilization, which has surpassed the defined threshold of 20%. The important aspects to note here are the following:

- **TARGETS:** Indicates the current/average CPU utilization (20%) against the target set (20%). The match here triggered the scaling.
- **MINPODS** and **MAXPODS:** Represent the minimum (2) and maximum (5) number of pods that HPA can scale to. In this case, the number of replicas has increased but is still within the defined limits.

- **REPLICAS:** Shows the current count of replicas (3), which has increased from the original count to handle the higher load.

5. HPA clean-up, delete HPA, and the deployment:

```
kubectl delete hpa webapp  
kubectl delete deploy webapp
```




Lab 3 - Installing KEDA and Setting Up Cron Scaler

Overview:

This lab provides practical experience of installing KEDA in a Kubernetes environment. We will make use of the Cron ScaledObject to scale the application based on a time schedule.

Prerequisites:

Kubernetes cluster with metric server installed as per Lab 1.

Exercise 3.1: Install KEDA Using Helm Chart

In this exercise, we will install KEDA using a Helm chart.

1. Install KEDA using Helm.

Execute the following commands, to get started:

```
helm repo add kedacore https://kedacore.github.io/charts
helm repo update
helm upgrade -i keda kedacore/keda --namespace keda --create-namespace
```

2. Verify KEDA pods are running in the cluster, using the command below:

```
kubectl get deployment -n keda
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
keda-admission-webhooks	1/1	1	1	13m
keda-operator	1/1	1	1	13m
keda-operator-metrics-apiserver	1/1	1	1	13m

3. Create a deployment, using the command below:

```
kubectl create deploy myapp --image nginx --replicas=2
```

4. Verify the deployment:

```
kubectl get deployments
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
myapp	2/2	2	2	37s

5. Create a **cron.yaml** file with contents below. This file defines a Kubernetes ScaledObject for your sample application.

```
apiVersion: keda.sh/v1alpha1
kind: ScaledObject
metadata:
  name: cron-scaledobject
  namespace: default
spec:
  scaleTargetRef:
    name: myapp
  triggers:
  - type: cron
    metadata:
      timezone: Asia/Kolkata
      start: 30 * * * *
      end: 45 * * * *
      desiredReplicas: "10"
```

This specification describes the **cron** trigger that scales workloads in/out based on a [Cron](#) schedule.

- **timezone** - One of the acceptable values from the Internet Assigned Numbers Authority (IANA) Time Zone Database. The list of timezones can be found in the Wikipedia article [“List of tz database time zones”](#).
- **start** - Cron expression indicating the start of the Cron schedule.
- **end** - Cron expression indicating the end of the Cron schedule.
- **desiredReplicas** - Number of replicas to which the resource has to be scaled between the start and end of the Cron schedule.

NOTE: Start/end supports “Linux format Cron” (MIN for minutes, HOUR for hours, DOM for day of the month, MON for month, and DOW for day of the week).

6. Create the ScaledObject using the below command:

```
kubectl apply -f cron.yaml
```

7. Verify the ScaledObject:

```
kubectl get scaledobject.keda.sh
```

NAME	SCALETARGETKIND	SCALETARGETNAME	MIN	MAX	TRIGGERS
AUTHENTICATION	READY	ACTIVE	FALLBACK	PAUSED	AGE
cron-scaledobject	apps/v1.Deployment	myapp			cron
True	False	Unknown	Unknown	75s	

8. The ScaledObject will scale the application pods to the desired number of replicas when the schedule triggers.

```
watch kubectl get all
```

NAME	READY	STATUS	RESTARTS	AGE
pod/myapp-747784665b-58vq7	1/1	Running	0	7m38s
pod/myapp-747784665b-5k8mg	1/1	Running	0	7m53s
pod/myapp-747784665b-92c47	1/1	Running	0	7m23s
pod/myapp-747784665b-cc298	1/1	Running	0	7m53s
pod/myapp-747784665b-fksp8	1/1	Running	0	7m38s
pod/myapp-747784665b-lhphb	1/1	Running	0	7m23s
pod/myapp-747784665b-mz2jd	1/1	Running	0	7m38s
pod/myapp-747784665b-tcvqq	1/1	Running	0	7m38s
pod/myapp-747784665b-vggttd	1/1	Running	0	7m53s
pod/myapp-747784665b-zdhv9	1/1	Running	0	7m53s

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
service/kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	49m

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
deployment.apps/myapp	10/10	10	10	13m

NAME	DESIRED	CURRENT	READY	AGE
replicaset.apps/myapp-747784665b	10	10	10	13m

NAME	TARGETS	MINPODS	MAXPODS	REPLICAS	AGE	REFERENCE
horizontalpodautoscaler.autoscaling/keda-hpa-cron-scaledobject	1/1 (avg)	1	100	10	10m	Deployment/myapp

```
press ctrl + c to exit
```

The Cron scaler allows you to define a time range in which you want to scale your workloads up and down. When the time window starts, it will scale from the minimum number of replicas to the desired number of replicas based on your configuration.

In this lab exercise, we made use of a simple Cron scaler, which scales the applications based on a specific schedule. You can further explore different scalers available.